Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Workstation Setup Guide: Local Lab Environment

Apache Kafka Core, Confluent Developer & Apache Spark Training

Target Environment: Local laptop / desktop (Windows 10/11 with WSL 2, Ubuntu Linux, or macOS)
No Cloud Accounts Required: Everything runs 100% locally on your machine.

1. Hardware & Operating System Minimum Requirements

Component	Minimum Requirement	Recommended for Optimal Speed	Why
RAM	16 GB Physical RAM	16 GB to 32 GB (Allocate ≥ 8 GB to Docker)	The 3 Kafka brokers, Schema Registry, Connect, ksqlDB, Postgres & Spark run concurrently.
Disk Space	30 GB Free Space	40+ GB SSD	Docker images take ~5 GB; Kafka logs and Spark shuffle data require disk buffer.
CPU	4 Cores (8 Threads)	8 Cores	Spark parallel worker tasks run locally.
Operating System	Windows 10/11 with WSL 2, macOS 13+, or Ubuntu 22.04+	Ubuntu 22.04 LTS or Windows 11 with WSL 2	Docker Compose and shell scripts require a Linux terminal.

2. Software Prerequisites Installation

Option A: Windows 10 / Windows 11 (WSL 2 Setup)

> **Important:**
> All lab commands must run inside the **Ubuntu terminal** in WSL 2, never in Windows PowerShell or CMD!

1. **Install WSL 2 with Ubuntu:**

   Open PowerShell as Administrator and run:

   ```
   powershell
   ```

   ```
   wsl --install -d Ubuntu-22.04
   ```

   Restart your computer if prompted. Set your username and password when Ubuntu opens.

2. **Install Docker Desktop for Windows:**

   * Download and install **Docker Desktop** from docker.com.

   * In Docker Desktop Settings:

   - Navigate to **General** → Check **"Use the WSL 2 based engine"**.
   - Navigate to **Resources** → **WSL Integration** → Enable integration for your `Ubuntu-22.04` distro.
   - Navigate to **Resources** → **Advanced** (or `.wslconfig`) → Set Memory to **8 GB minimum** (10 GB recommended) and CPUs to **4**.

3. **Keep the Lab Folder inside the Linux Filesystem:**

   Inside your Ubuntu terminal, work under your home directory:

   ```
   bash
   ```

   ```
   mkdir -p ~/labs && cd ~/labs
   ```

   Do NOT place your lab folder under `/mnt/c/` *!* Cross-filesystem I/O between Windows and Linux is slow and breaks file permissions.

Option B: Ubuntu Linux (22.04 / 24.04 LTS)

Run the following in your terminal:

```
# 1. Update package lists
sudo apt update && sudo apt upgrade -y

# 2. Install Docker Engine and Docker Compose Plugin
sudo apt install -y ca-certificates curl gnupg lsb-release
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/key
rings/docker.gpg
echo "deb [arch=⟨!—KATEX_INLINE_6—⟩(lsb_release -cs) stable" | sudo tee /etc/apt/sources.l
ist.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compos
e-plugin

# 3. Add your user to the docker group (avoids needing sudo)
sudo usermod -aG docker $USER
newgrp docker

# 4. Install Python 3, venv, and Java 17
sudo apt install -y python3 python3-pip python3-venv openjdk-17-jdk-headless git
```

Option C: macOS (Apple Silicon M1/M2/M3 or Intel)

1. Install **Docker Desktop for Mac** from docker.com.
   * In Settings ⟶ Resources ⟶ Set Memory to **8 GB**, CPUs to **4**.
2. Install tools using **Homebrew**:
   `bash`
   `brew install openjdk@17 python@3.11 git`
   `sudo ln -sfn /opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk-17.jdk`

---

3. Ports Required on Your Workstation

Ensure the following local ports are not occupied by existing local databases or web servers:

Port	Service	Used In
9092	Kafka Broker 1 (External Listener)	All Sessions (S01 to S12)
9093	Kafka Broker 2 (External Listener)	Cluster Replication & Broker Failure (S02)
9094	Kafka Broker 3 (External Listener)	Cluster Replication & Broker Failure (S02)
8081	Confluent Schema Registry REST API	Schema Contracts & Avro (S05)
8083	Kafka Connect Distributed REST API	Data Ingestion Connectors (S06)
8088	ksqlDB Streaming SQL Engine	Real-time Stream SQL (S07)
5432	PostgreSQL Database	Source Data for Kafka Connect (S06)
8080	Kafka UI (Web Console)	Visual Inspection Dashboard (All Sessions)

---

4. Initializing the Local Lab Stack

Once your terminal is open in the `00_Lab` directory:

```
# 1. Start the complete multi-broker cluster in the background
make up

# 2. Run the automated pre-flight healthcheck
make check
```

You should see 6 green PASS indicators:

```
PASS  kafka cluster (3 brokers reachable)
PASS  schema registry
PASS  kafka connect
PASS  ksqldb server
PASS  postgres
PASS  kafka ui
```

To access the visual web dashboard, open your browser to:
👉 `http://localhost:8080`

Enterprise Real-Time Data Engineering · Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me) · Course Materials: fakhr.me/KafkaSpark · Repository: github.com/fakhruldeen/KafkaSpark